

VC Attention

Fusedpipe and D Scheduling

B200 / SM100 | Technical report | 4 October 2026

Two incremental scheduling optimizations for an existing dense FP8 attention kernel. Fusedpipe exposes independent reads in CuTe source. D changes dependency control and local max scheduling in the compiled SASS. This report separates their matched measurements from the larger benefit of the complete VC attention algorithm.

Layer	Change	Real-capture increment
Existing VC algorithm	FP8, ExpCast, Tensor Core denominator and inline output rescaling form the common base.	Not decomposed here
Fusedpipe	Read the current K descale earlier; replace four 32-column output correction fragments with two preloaded 64-column fragments.	1.013014x versus pre-change VC
D	Assign an independent dependency slot to the first score load; move two existing max leaves before the full-read completion point.	1.002647x versus fusedpipe

Both optimizations schedule existing work more effectively. Neither adds warps, removes attention interactions, changes the softmax reduction tree, nor introduces a new approximation. Byte equality is established against the matching FP8 control, not against BF16.

Scope of the main results

One noncausal sequence, equal query/KV head counts, D=128 and S=73,426 on NVIDIA B200. Inputs are privately captured MiniMax-H3 attention tensors from step 24, layers 0 and 24. H=7 selects heads 0-6 from H=56 captures; it is not a live context-parallel execution. All Q/K/V descales are supplied, mid_window_blocks=4, and all skipping is disabled.

Timing is kernel-only: one attention kernel replayed under CUDA-event timing. Input quantization, V packing, compilation, allocation, communication and the rest of model inference are excluded. The two incremental ratios above come from separate 20-round paired cohorts and must not be multiplied into a new measured result.

Release policy: since 7ae8e21, the public default is mid4 with eligible B200 fusedpipe; D is disabled. Historical measurements below retain revision cbb2da8. Pages 13-14 document the public reproduction tools and recovered environment.

Section 10 adds a separate four-case, fixed-seed synthetic sweep at S=188,214 and 262,144. It includes freshly paired BF16 timings. D has a small median regression in one case; the new sweep does not establish a uniform D benefit. Raw paired samples and seeds are supplied in the companion JSON.

01 Mathematics and pipeline context

Let r_{ij} be a raw FP8 Q-K dot product. dQ_i denotes the Q block descale for row i , dK_b the current K block descale, and $\sigma = 1/\sqrt{D}$. D is the per-head Q/K dimension, 128 here. For this scaled path, the ideal restored logit is $\sigma * dQ_i * dK_b * r_{ij}$.

```
c_i      = sigma * dQ_i * log2(e)
tilemax  = dK_b * max_j(r_ij)
alpha_i  = 2 ** (c_i * (M_i - M_i_new))
A_new    = alpha_i * A_old + P_block @ V_fp8
l_new    = alpha_i * l_old + sum(P_block)
O_final  = dV * A_final / l_final
```

A is the unnormalized output numerator and l is the denominator. Kernel code also calls the TMEM numerator accumulator O ; "O rescale" below refers to this accumulator, not a previously normalized final output. The equations describe online-softmax scaling. Actual P uses the existing FP8 ExpCast encoder, Tensor Core denominator and max deadband policy, so this is not exact BF16 softmax.

Why numerator and denominator must both be rescaled

When the running maximum changes, prior probabilities are expressed relative to an older exponent origin. Multiplying both accumulators by the same α preserves their relationship. Fusedpipe changes when the values are loaded and how columns are grouped; it retains the same α and per-element multiply.

Role	Existing work	Change in this report
Softmax warps 0-7	Read score TMEM, reduce row maxima, encode P and perform the established inline correction work.	Internal scheduling only
Other CTA roles	Load, MMA and service roles; the CTA has 12 warps / 384 threads.	No new warps or participation changes
Producer-consumer handoff	S_{full} publishes scores; $P_{full_O_rescaled}$ publishes first P and corrected O .	Protocol retained

K descale scales the tile maximum and the raw-score encoder multiplier. The ExpCast bias is first formed using the base code scale and the selected row maximum; K descale is not multiplied into that bias a second time. These operation and rounding choices are unchanged.

[S1] Warp roles and register allowances (line 807)

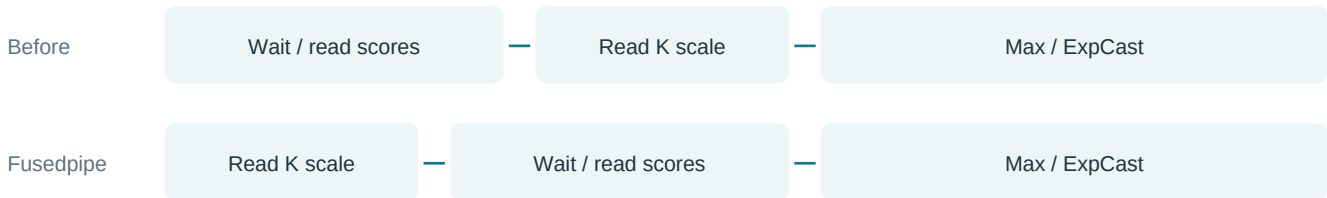
[S2] ExpCast scale and bias construction (line 3907)

[S3] Existing maximum and rescale update (line 213)

TECHNICAL REPORT / 03

02 Fusedpipe: read K descale earlier

The original path waits for QK scores, reads and releases the score buffer, applies masking, and then reads the current K block descale. Fusedpipe moves that same immutable scalar read before the S_full wait. Its batch, KV-head and block indices are already available.



Logical issue order only. Box widths do not represent measured cycles.

```
# Before
wait(S_full[stage])
scores = load_score_TMEM()
wait_score_loads_and_arrive_score_empty()
apply_mask(scores)
k_scale = K_descale[batch, kv_head, n_block]

# Fusedpipe
k_scale = K_descale[batch, kv_head, n_block]
wait(S_full[stage])
scores = load_score_TMEM()
wait_score_loads_and_arrive_score_empty()
apply_mask(scores)
```

What can overlap

The scalar load does not depend on the QK result. It can progress while the softmax warp waits for S_full. In canonical mid4 SASS, LDG.E at 0xa6f0 precedes the score-ready TRYWAIT at 0xa710. This is the current n_block, not speculative prefetch of the next block.

Why the lifetime remains valid

The index, value, number of reads and consumption sites are unchanged. Score TMEM loads remain after S_full, and score_empty remains after score-load completion. The descale is an input scalar, not a producer-owned score buffer being overwritten by MMA.

This transformation moves a logical load; it neither adds preprocessing nor reduces its logical data payload. Cache and transaction behavior affect physical traffic. The measured fusedpipe increment includes both source changes and any resulting compiler scheduling differences; it is not an isolated measurement of this LDG.

[S4] Eligibility, early descale load and unchanged S_full wait (line 3665)

[S5] Original descale load retained as a fallback (line 3792)

03 Fusedpipe: preload two O halves

The original correction helper handles four 32-column fragments as read-multiply-write chains. The new helper issues two independent 64-column loads before the first dependent multiply. The second read can progress while the first half is multiplied and stored.

```
# Before
for cols in [0:32, 32:64, 64:96, 96:128]:
    x = load_TMEM(0[cols])
    store_TMEM(0[cols], x * alpha)

# Fusedpipe
a = load_TMEM(0[0:64])
b = load_TMEM(0[64:128])
store_TMEM(0[0:64], a * alpha)
store_TMEM(0[64:128], b * alpha)

# Unchanged: denominator remains LAST
l = load_TMEM(denominator)
store_TMEM(denominator, l * alpha)
fence_TMEM_stores()
```

Canonical PC	Operation	Meaning
b210 / b230	LDTM.x64 / LDTM.x64	Both O reads precede consumption
b250	First FMUL2	First dependent O multiply
b470 / b660	STTM.x64 / STTM.x64	Store the two corrected halves
b670 / b680 / b690	LDTM / FMUL / STTM	Denominator remains after both O halves
b6a0 / b6e0	FENCE / ARRIVE	Publish only after correction stores

For each logical row that executes correction, the payload remains 128 FP32 O elements plus one FP32 denominator: **516 bytes read and 516 bytes written**. These are logical TMEM bytes, not HBM transactions. No additional Q/K/V TMA transfers or TMEM allocation are introduced.

First tiles do not read uninitialized O. The original warp ballot still controls rescaling; no-rescale warps do not perform new unconditional O reads. The first P segment is stored before correction but is not published to MMA until the correction fence. Source pseudocode is not a promise of every final SASS issue order.

[S6] [Two-half helper; denominator still read last \(line 4734\)](#)

[S7] [Conditional correction and P publication \(line 4027\)](#)

04 D: use the first score segment sooner

D modifies two repeated softmax bodies in the fusedpipe binary. Each body changes five existing 16-byte instruction slots: ten slots and 74 changed bytes in total. The issue PCs of all four score LDTM.x32 instructions remain fixed. Two existing max leaves move into earlier slots, while a uniform-register move and a NOP move later.

Body 1 PC	Original fusedpipe	D
a750	LDTM.x32 R36, tmem[UR10]	Same load; add write dependency B1
a760-a780	Other three score loads; final load writes B0	All instruction words unchanged
a790	R2UR UR9, R0	FMNMX3 R77,R36,R37,R38,!PT; wait B1
a7a0	NOP	FMNMX3 R76,R39,R40,R41,!PT
a7b0	score_empty ARRIVE; wait B0	Same PC and complete instruction word
a7c0	First max leaf, producing R77	R2UR UR9,R0
a7d0	Second max leaf, producing R76	NOP

Load granularity and wait granularity are different

The first x32 load produces R36-R67. The next three produce R4-R35, R164-R195 and R132-R163. The two moved max leaves consume only R36-R41. Assigning a separate dependency slot to the first load allows those leaves to wait for that segment rather than for all four segments.

```
Before: LD0 LD1 LD2 LD3 | R2UR NOP | WAIT_ALL/ARRIVE | MAX0 MAX1
D:      LD0 LD1 LD2 LD3 | WAIT_0/MAX0 MAX1 | WAIT_ALL/ARRIVE
                                     | R2UR NOP
```

B0/B1/B2 identify hardware dependency scoreboard slots, not shared-memory mbarrier indices. "Write B1" records an asynchronous result dependency; it does not write application data to memory. The original full-read wait still constrains score_empty publication.

D does not insert max instructions between score loads, omit a score load or remove completion. It makes two existing max leaves available after the first segment is ready, while later segments may still be returning. This is a dependency-level scheduling explanation, not a measured per-instruction cycle attribution.

[S8] [Complete ten-slot patch table \(line 25\)](#)

05 D: second body and safety checks

Body 2 PC	Original fusedpipe	D
dea0	LDTM.x32 R36, tmem[UR6]	Same load; add write dependency B2
deb0-ded0	Other score loads; final load writes B1	All instruction words unchanged
dee0	R2UR UR5, R0	FMNMX3 R76,R36,R37,R38,!PT; wait B2
def0	NOP	FMNMX3 R75,R39,R40,R41,!PT
df00	score_empty ARRIVE; wait B1	Same PC and complete instruction word
df10 / df20	Original positions of the two max leaves	R2UR UR5,R0 / NOP

Dependency slots cannot be copied blindly between bodies

In the second body, B1 already protects the full score read. B0 also protects the descale load at 0xde40 and is waited on before its related overwrite at 0xdf70. D therefore uses retired B2. Its availability was checked through the entry waitmask at 0xde60 and the slow-wait return path, not just the adjacent instructions.

Review item	Established boundary
Arithmetic	The same source registers, !PT modifier and ordered max tree remain. Later combining nodes at a880 / dfd0 are unchanged.
Register conflicts	The reordered instruction pairs have no RAW, WAR or WAW conflict; max results do not overwrite raw-score operands.
Delayed uniform moves	UR9 / UR5 are still initialized before their first later consumer at b1b0 / e8f0.
Dependency retirement	Review includes entry and slow-wait control flow, B1 in body 1, B2 in body 2 and the preserved B0 dependency.
Completion and metadata	All 33 ARRIVE PCs and instruction words, including associated 0x3904 metadata records, remain unchanged.

Keeping ARRIVE at the same PC does not mean it executes in the same cycle. Moving useful work near that wait is the intended optimization. Code length, branch targets, register-allocation metadata and the memory-access set do not grow.

Full machine-code differences, control-flow/liveness review and byte-equal outputs on matching FP8 inputs support the checked binary family. They are not a proof for arbitrary compiler output. A new layout must pass the complete runtime guard rather than a short instruction-pattern match.

06 Matched incremental performance

B200, S=73,426, D=128. L00 / L24 identify layer 0 / 24 of step 24. Each row is a separate 20-round paired test. Speedup = median(control) / median(candidate). The aggregate is the equally weighted geometric mean of four case ratios; wins count rounds in which the candidate was faster.

Fusedpipe versus pre-change experimental VC: 1.013014x

Case	Control ms	Fusedpipe ms	Speedup	Wins
I00-h7	6.451459	6.401475	1.007808x	17/20
I00-h56	56.680652	55.950128	1.013057x	20/20
I24-h7	6.392039	6.287898	1.016562x	20/20
I24-h56	56.353769	55.540204	1.014648x	20/20

An independent 12-round cohort measured 1.014248x with 48/48 wins; the 20-round cohort had 77/80 wins. The repeated aggregate speedup is approximately 1.30%-1.42% above 1x for these cases. Cohorts are not pooled, and the better batch is not selected per shape.

D versus already-fusedpipe: 1.002647x

Case	Fusedpipe ms	With D ms	Speedup	Wins
I00-h7	6.365821	6.344682	1.003332x	13/20
I00-h56	55.842206	55.718091	1.002228x	17/20
I24-h7	6.266525	6.244125	1.003587x	14/20
I24-h56	55.224094	55.144452	1.001444x	12/20

The first 12-round cohort measured 1.002887x and 37/48 wins; confirmation had 56/80 wins. The 0.26%-0.29% repeated aggregate signal is small. It does not establish a uniform benefit at every shape or time interval.

What the measurements do and do not attribute

CUDA events measure five replays of one attention kernel per timing sample, after two warm calls and a one-second warmup. Preparation and validation are outside timing. Included cases completed with clean ownership guards; clocks were not locked. Profiling duration is not used as performance data.

Fusedpipe is measured as one combined source change; these results do not assign separate percentages to K prefetch and O preloading. D already has fusedpipe in its control. Ratios from the two cohorts must not be multiplied and presented as a newly measured combined speedup.

07 BF16 context, resources and correctness

Historical whole-VC comparison: approximately 2.137x

The pinned BF16 reference uses original BF16 QKV and its default scan order. VC uses preprocessed FP8 inputs, all descales and mid4. BF16 medians are from a separate 20-round cohort on 2 October; D medians are from its 3 October confirmation. This table is contextual arithmetic, not a newly paired BF16/D benchmark.

Case	BF16 ms	VC + D ms	Historical ratio
I00-h7	13.656083	6.344682	2.152367x
I00-h56	114.823511	55.718091	2.060794x
I24-h7	13.896109	6.244125	2.225469x
I24-h56	116.539191	55.144452	2.113344x

The geometric mean is 2.137153x. This includes the entire VC algorithm and earlier optimizations. It is neither the isolated D gain nor an end-to-end model speedup. The BF16 source revision is 2cae9072801704491b37f14037b4baa32c3958dc; provenance is retained in the public vendored-source manifest.

Registers: allocation and liveness are distinct

Metric	Fusedpipe -> D	Interpretation
REG / STACK metadata	168 / 8 -> 168 / 8	Initial resource fields, not dynamic live values
Softmax USETMAXREG	232 -> 232	Role-specific allowance, not measured consumption
Conservative CFG live peak	205 -> 205	Static analysis, not a hardware register-use counter
Static LDL / STL count	3 / 2 -> 3 / 2	Existing epilogue spills; no new hot-loop spill
Text bytes / instruction slots	87,040 / 5,440 -> unchanged	Existing slots are rearranged

For the matched frozen pre-change VC and fusedpipe binaries, conservative static GPR peak rises from 169 to 205, with unchanged initial allocation and role allowance. The larger lifetime is consistent with both O halves being live, but the difference is not a dynamic measurement of 36 extra registers and does not by itself establish occupancy changes. SHARED=1024 excludes dynamic shared memory.

Correctness scope

Scheduling candidates are compared byte-for-byte with matching FP8 controls. The scan-window release validation passed ten cases and 120 complete-output comparisons. Equality for these changes does not imply FP8 equals BF16, exhaust all inputs, or establish model-quality equivalence.

[S9] [Public baseline and implementation provenance \(line 1\)](#)

08 Applying D to the actual callable

D is applied per compiled function after `cute.compile()`. The adapter exports a host object, identifies its embedded CUDA ELF, validates it, rewrites approved instruction words in memory, and binds the same ABI through `BinaryExecutionEngine` and `TVM FFI`. It retains the required objects and lifetimes. It does not install a global driver hook or mutate the original cubin or compiler on-disk cache; the API cache stores the wrapped callable.

```
cute.compile(...)
-> export host object
-> locate the single embedded CUDA ELF
-> validate architecture, text family and metadata
-> rewrite 10 existing instruction slots
-> bind the same function ABI
-> cache the wrapped callable

unsupported layout -> keep the original callable
```

Guard	Constraint
Architecture and adapter	SM100 flags; ELF64 little-endian; CuTe DSL 4.6.0 and the supported FFI wrapper
Complete native code	A single target text section, 87,040 bytes; complete normalized pre/post SHA-256 validation
Scan-window fields	Only four proven immediate fields may vary; each must equal the caller-supplied window
Execution metadata	Complete canonical fingerprint, with only explicitly defined naming normalization
Mutation boundaries	All ten old words must match; post-patch length and target hash must match

The revision is `sm100-score-two-body-window-v2`. Instance metadata records the actual and normalized text hashes, window and applied status. A Python version label alone is not proof of the loaded code; external AOT loaders or bypass paths need their own loaded-artifact verification.

Unsupported layouts retain the original callable. Errors from actual kernel execution are not silently swallowed as a patch mismatch. The `export_to_c` wrapper also revalidates the patch with its bound scan window.

[S10] [Window normalization and complete native-code validation \(line 142\)](#)

[S11] [Per-callable loading adapter \(line 1\)](#)

09 Eligibility and specialization boundaries

Sequence length alone is not an adequate eligibility rule. A parameter can select a different compiler branch, affect alignment or ABI, change resource allocation, or remain runtime data within one specialization. The complete code and metadata guard remains authoritative.

Factor	How it may change the compiled layout
Scan window	A compile-time scan constant. Common values change four immediate fields; 0/1 produce different layouts and currently fall back for D.
Sequence lengths, heads, batch	Can cross Q-stage, packing, split-KV or stride/layout dispatch thresholds; within a branch they may remain runtime parameters.
D / Dv, tiles, stages, warps	Change unrolled work, registers, TMEM/SMEM layouts and synchronization participants.
Dtype and descale presence/rank	Select different arithmetic and loading paths; scaled and unscaled binaries are not interchangeable.
Mask, score modifier, skip, LSE	Add control flow, output or side effects. GQA, paged KV and alternate quantization can select other specializations.
Architecture and compiler	Target architecture, DSL, ptxas and compilation options may change instructions and dependency control.

Current checked family

CPU native-family checks cover windows 2, 3, 4, 7, 8, 16, 31 and 1024. D falls back for 0, 1 and None. The fusedpipe source gate accepts non-None nonnegative windows, so 0/1 can use fusedpipe without D. The source path is limited to the checked SM100 schedule, E4M3 Q/K/V, all descales with rank-3 K descales, Q-stage 2, 128x128 tiles, D/Dv=128, inline correction and first-P64 handoff; skipping and custom score/mask modifiers are excluded.

CPU mid8 shape evidence covers [32769,7,128], [73426,7,128], [73426,56,128] and [188214,7,128]. This is not their full Cartesian product, and compilation evidence alone is not a long-sequence performance result.

The ten-case window release compared two public revisions. At windows 2/8/16/1024, enabling both fusedpipe and D is a combined change, not isolated D. At None and 4, both sides selected identical native text; their small timing differences are regression noise, not a new optimization gain.

- [S12] [Exact source-level fusedpipe gate \(line 3668\)](#)
- [S13] [Runtime D attempt followed by strict binary validation \(line 1283\)](#)

10 Longer-sequence extension

Fixed-seed synthetic inputs, separate from the real-capture results. Independent CPU torch.randn BF16 Q/K/V use seeds 20261004, 20261005 and 20261006. One noncausal sequence, D=128, equal Q/KV heads, all descales, mid4 and no skipping. These are newly generated long tensors, not padded or repeated model captures.

Four matched implementations: median latency in milliseconds

[S,H,D]	Pre-fusedpipe VC	Fusedpipe	Fusedpipe + D	BF16
[188214, 7, 128]	49.304358	48.244330	48.137207	94.809500
[188214, 56, 128]	402.057336	394.094482	393.837146	763.531982
[262144, 7, 128]	96.323181	94.386279	94.313400	185.550647
[262144, 56, 128]	781.894336	771.211157	772.152124	1480.788721

Separate ratios, each computed within the same run

[S,H,D]	Fusedpipe increment	D increment	Whole VC vs BF16
[188214, 7, 128]	1.021972x 12/12 wins	1.002225x 9/12 wins	1.969568x 12/12 wins
[188214, 56, 128]	1.020205x 12/12 wins	1.000653x 8/12 wins	1.938700x 12/12 wins
[262144, 7, 128]	1.020521x 12/12 wins	1.000773x 7/12 wins	1.967384x 12/12 wins
[262144, 56, 128]	1.013852x 12/12 wins	0.998781x 8/12 wins	1.917742x 12/12 wins

Ratios are pre-fusedpipe/fusedpipe, fusedpipe/D and BF16/D, respectively. Wins count faster candidate rounds; a ratio of medians and a paired win count describe different aspects of the same samples. The BF16 numerator is freshly measured in the same run, not reused from the historical table.

Equal-weight geometric means across these four synthetic cases: **1.019133x** for fusedpipe, **1.000607x** for D, and **1.948230x** for complete VC + D versus BF16. These are kernel-only measurements, not an inference-pipeline speedup or a prediction for real-model inputs.

D does not improve every case. At [262144,56,128], D takes 0.122% longer by the ratio of medians, despite winning 8/12 paired rounds. These summaries can disagree because they aggregate samples differently. This is a tiny regression in one cohort, not evidence of a stable general regression or improvement.

This is one fixed-seed synthetic sweep with no independent repeat cohort. In particular, the small D aggregate must not be described as a stable universal benefit. The real-capture cohort and its historical BF16 ratio remain separate.

10 Long-sequence method and evidence

Item	Recorded boundary
Input generation	Independent CPU torch.randn BF16 Q/K/V; Q/K/V seeds 20261004 / 20261005 / 20261006. Equal Q/K/V head counts; S=188214 or 262144 and H=7 or 56.
Quantized configuration	All Q/K/V descales supplied, ExpCast, packed V, inline O rescale, Tensor Core denominator, mid4 and no skipping.
Timing	12 complete paired rounds per case; five kernel replays per sample, two warm calls, one-second warmup. Each implementation occupies each order position three times.
Excluded work	QKV generation and quantization, V packing, allocation, compilation, validation, profiling, model execution and communication.
Correctness and ownership	All four accepted cases passed complete scheduling-output byte comparisons and single-kernel checks. Interruptions and foreign-process contamination invalidate an entire attempt.
Clock and repetition limits	Clocks were not locked. One accepted cohort per shape; no independent repeat cohort.

Exactly which implementation was tested

The four routes are a matched pre-fusedpipe VC control, fusedpipe, its exact D binary derivative, and a pinned BF16 reference freshly timed in the same run. The controls are frozen experimental kernels, not unmodified current public checkouts. The historical H7 dispatch adaptation is shared by the relevant experimental controls.

Only three attention kernel records were needed for BF16, pre-fusedpipe VC and fusedpipe. The four long shapes share the respective native functions with different runtime dimensions; D derives from the fusedpipe artifact. The fusedpipe text and D guard match the canonical 87,040-byte family. Correct execution at S=262144 is new evidence from this typed-AOT replay path, not proof that every public-API configuration is supported.

CPU artifact verification, isolated host/internal symbols and invocation of a single-kernel callable identify the tested D derivative. This sweep has no new device-code dump. Complete FP8 scheduling-output equality is required; equality to BF16 and model-quality equivalence are not claimed.

Public supplementary results

```
VC-Attention-Long-Sequence-Results.json
```

The companion JSON contains every accepted raw paired timing sample, generation seeds and input hashes, configurations, per-case medians, wins, ratios, order-position counts and artifact hashes. It includes no private model input paths or infrastructure hostnames. Rejected attempts are excluded from every table and aggregate; their count is recorded.

11 Public reproduction workflow

The public tools in `tools/report_reproduction` rebuild isolated controls from this checkout. They create a private source tree and CPU AOT artifacts; the installed inference package and its defaults are unchanged.

Route	Construction and identity
<code>pre_fusedpipe</code>	Set only <code>use_fusedpipe=False</code> in the staged v4 source. Keep <code>mid4</code> , all descales and the shared B200 packing gate.
<code>fusedpipe</code>	Clone the same v4 source under an isolated namespace; keep the <code>fusedpipe</code> schedule enabled.
<code>bf16</code>	Use the pinned baseline snapshot with original BF16 inputs.
<code>d (explicit opt-in)</code>	Derive from the exact <code>fusedpipe</code> object. Check canonical text/metadata and isolate host symbols; fail if the fingerprint differs.

The report control `pre_fusedpipe` is not the public CLI backend `vc_v4`: that historical CLI name selects the original scan, whereas both report FP8 controls use `mid4`. Native equivalence is identified by artifacts, not labels.

```
python tools/report_reproduction/build.py --output build/report
python tools/report_reproduction/benchmark.py \
  --build build/report --shape 32769x7x128 \
  --output results/report-smoke.json
python tools/report_reproduction/summarize.py \
  results/report-smoke.json
```

Long-sequence matrix and D

Repeat the runner for $S=188214/262144$ and $H=7/56$, $D=128$. Defaults are seeds 20261004/5/6, 12 balanced rounds, 5 replays, 2 warm calls and 1 second of warmup. Pass `--include-d` to both build and benchmark only for the experimental fourth route. A D guard rejection is an error, never a relabeled `fusedpipe` fallback.

Exact timer boundary

Input generation, quantization and V packing happen before capture. The runner captures the native AOT call, checks direct/graph output bytes and profiles exactly one attention kernel outside timing. Complete post-timing byte checks, artifact hashes and raw paired samples are retained. The ordinary `vc-attn-bench` scope includes V packing and remains a separate operator benchmark.

See `docs/reports/reproduction.md` for complete commands, prerequisites, expected output and failure handling. Fresh measurements form a new cohort; the published historical tables are unchanged.

12 Environment and validation boundaries

Recovered historical long-sequence environment

Component	Recorded value
GPU / driver	NVIDIA B200 / SM100; driver 590.48.01
Python / PyTorch	Python 3.12.3; PyTorch 2.11.0+cu130; CUDA runtime 13.0
DSL / Triton / quack	CuTe DSL 4.6.0; Triton 3.6.0; quack 0.6.1
CUDA Python / linker	cuda-python 13.0.3; GCC 13.3.0; GNU objcopy 2.42
Clock policy	Clocks were not locked. Per-case start/end clock, pstate and power snapshots are now public.

historical-environment.json recovers these fields from the original accepted records and retains their hashes. The historical ptxas binary identity and complete compiler environment were not recorded. They remain explicitly unknown; current tool versions are not substituted for historical facts.

What a new build records

manifest.json records package versions, available compiler/linker identities, discovered bundled ptxas candidates, compile options, runtime library hashes, source transformations, ABI/cache metadata, artifact hashes and explicit D patch proof. Candidate compiler discovery does not itself prove which binary the DSL invoked. Rebuild and run in the same pinned environment.

Correctness, numerical approximation and ownership

FP8 scheduling controls must be finite and byte-equal; direct and graph calls must match, including after timing. BF16 is a timing reference with its own finite-output and direct/graph checks; FP8 equality to BF16 is not claimed. The runner checks foreign CUDA PIDs around preparation and every timed sample; reserve the GPU externally. Shared-GPU opt-in results are explicitly marked non-isolated.

Current release and experimental scope

The default API uses mid4 and eligible source-level fusedpipe, with D disabled. D remains an experimental offline replay option and may reject a different compiler family. This report does not establish general real-UInt8 FFI ABI compatibility, model-quality equivalence, backward support or end-to-end speedup. Current reproduction validation is recorded separately in reproduction-validation.json.

Private captured-input tensors are not distributed. The synthetic long-sequence matrix can be regenerated and input-hash checked. Its original single-seed cohort and small D effect do not establish a universal benefit.

Appendix Canonical patch and fingerprints

Canonical mid4, text-relative PCs. Each OLD/NEW value is a complete 128-bit instruction word, stored little-endian in ELF. All unlisted slots remain unchanged. The four scan immediate fields are normalized only for family validation; the caller's actual scan window is preserved.

a750	OLD	000fe400082c00000000000a002479ee
	NEW	000e6400082c00000000000a002479ee
a790	OLD	000fe200000e00000000000000972ca
	NEW	002fe4000780002600000025244d7276
a7a0	OLD	000fe2000000000000000000007918
	NEW	000fc4000780002900000028274c7276
a7c0	OLD	000fe4000780002600000025244d7276
	NEW	000fe200000e00000000000000972ca
a7d0	OLD	000fc4000780002900000028274c7276
	NEW	000fe2000000000000000000007918
dea0	OLD	000fe400082c00000000006002479ee
	NEW	000ea400082c00000000006002479ee
dee0	OLD	000fe200000e00000000000000572ca
	NEW	004fe4000780002600000025244c7276
def0	OLD	000fe2000000000000000000007918
	NEW	000fc4000780002900000028274b7276
df10	OLD	000fe4000780002600000025244c7276
	NEW	000fe200000e00000000000000572ca
df20	OLD	000fc4000780002900000028274b7276
	NEW	000fe2000000000000000000007918

Normalized SHA-256 fingerprints

before	3f8c9a6f08115780f331d674a3efffa774
	10d88bd2fe9ee91c9fffeed74367591
after	040efd30afd2544024a7a36046ad8382d
	4687d2d3d6ce6da9fee4d60b0b6755f
meta	9ab3d51a5555fc0a52621a3623fe36dd
	d0fa20d1a1802bfd5765ac6e23946e42

Scan immediates: PCs 1740, 5fc0, 7ca0 and a570; only bits 32-63 of each word participate in window normalization. These PCs identify the canonical binary family, not universal addresses for other compiler versions.